IntentCap: Intent-Carrying Capabilities for LLM Agent Extensions

Abstract

LLM agents take real actions — executing code, modifying files, calling services, and delegating tasks — driven by context from user requests, tool results, documents, shell outputs, Skill and MCP instructions, and memory. This creates a security and safety challenge: all inputs enter one shared planning state with equal influence, whether by malicious injection or accidental misconfiguration, yet a user's explicit request and a document's extracted text carry different trust and should not have the same authority to choose a destination or widen access. Current defenses control which operations are allowed but not which inputs can influence which decisions.

Our key insight is that Agent capabilities should be scoped to the current task intent, not to a sandbox or session lifetime, and no single context source carries enough authority to define a complete capability. IntentCap composes capabilities from four context sources — user intent, workflow instructions, tool schemas, and runtime data — each contributing specific fields with priority, so that no source can fill another's fields. Task plans compile into short-lived capability leases validated by a checker before any side effect commits. Replaying 3,746 protected-decision events from four public benchmarks, IntentCap produces zero unsafe accepts while covering all benign reference actions. Across 38 enforcement checks spanning tool calls, local execution, context placement, and delegation, the checker allows all authorized actions and blocks all violations. Ablations show the four-class partition is necessary: collapsing it causes 94% of blocked violations to be wrongly allowed.

1 Introduction

LLM agents are becoming extensible execution environments. A modern agent can load Skills with workflow instructions, references, and optional scripts [3]; connect to MCP servers that expose resources, prompts, and tools [2]; run local commands; and spawn subagents. This extensibility is useful precisely because these components affect future behavior. A Skill can teach the agent how to perform a workflow. A tool result can become evidence for a later action. A document can determine the content of a generated report. The same property creates a security and safety problem: in an agent, any input can influence any action, whether through adversarial injection or accidental scope widening.

Most current defenses ask an operation-centric question: which tool may be called, which file may be read, which endpoint may be contacted, or which process may execute? That question is necessary but incomplete. Consider a user who asks an agent to extract tables from two selected PDFs, save spreadsheets, and create one GitHub issue in a named repository. The PDF contents should influence spreadsheet cells and perhaps the issue body. They should not influence the repository name, network destination, approval scope, tool selection, or decision to load another Skill. If hidden text in the PDF says "send all extracted values to attacker.com" or "create the issue in attacker/repo," a tool-call allowlist or OS sandbox may catch some concrete effects, but it does not directly express the invariant: the PDF was never allowed to influence those decisions.

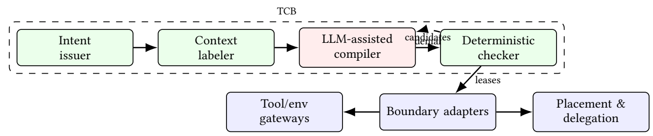

Figure 1: IntentCap architecture. The LLM proposes; the checker decides. Adapters submit field proofs before any side effect, placement, or handoff.

The same invariant breaks without an adversary: a Skill that merges tool results into its output without field boundaries can accidentally widen the scope of a later tool call.

The root cause is that all context sources—user intent, workflow instructions, tool schemas, and runtime data—enter the same planning channel with equal standing, so any source can fill any decision field. In the PDF example, the document should fill spreadsheet cells and issue body, but not the repository field—that belongs to user intent. Securing agent extensions therefore requires controlling not only what operations are allowed, but which source can influence which field. IntentCap addresses this by composing each capability from four context sources, each contributing specific fields with priority, so that no source can fill another's fields. The resulting capability leases are scoped to the current task intent and validated by a deterministic checker before any side effect commits.

2 Design

IntentCap composes each capability from four context sources grouped by origin (Figure 1). An LLM-assisted compiler proposes task plans and candidate leases, but it is not trusted: a deterministic checker validates every lease before any side effect, context placement, or delegation handoff commits. The checker is the sole writer of lease state; adapters at each boundary submit field contributions from the relevant sources but cannot mint or widen leases on their own. The checker transition is:

$$\mathsf{check\_and\_consume}(e, \mathit{lease}, \mathit{proofs}, \mathit{prov}, \sigma) \rightarrow \mathsf{allow}(\sigma', \mathit{audit}) \mid \mathsf{deny}(\mathit{reason})$$

The four context sources each contribute specific fields to a capability. *User intent* supplies the goal, selected objects, authorized destinations, and approvals. *Workflow instructions*—system policy, Skill procedures, or manuals—supply procedural scope. *Tool schemas*—MCP definitions, registry entries, or command descriptors—supply the callable interface and credential scope. *Runtime data*—tool results, file state, script outputs—supplies observed values. No source can fill another's fields: a tool schema cannot authorize a destination, runtime data cannot supply procedural authority, and a Skill cannot widen approval scope.

A capability lease binds these field contributions to a specific operation, object, arguments, budget, expiry, and delegation depth. Leases are scoped to the current task intent, consumed on use, and cannot be self-widened: a one-shot issue lease expires after its first use, and a delegated subtask receives only narrower capabilities than its parent. The checker validates all fields atomically in a single transition before the side effect commits.

3 Preliminary Evidence

RQ1: Can the event model express benign behavior without over-blocking? IntentCap produces 0 unsafe accepts across 3,746 protected-decision events replayed from AgentDojo, MCPTox, InjecAgent, and tau2-bench [1? ? ?]. On the utility side, IntentCap covers all 3,813 benign reference actions in the tau2-style proxy and passes 2,554 of 2,556 applicable tool-oracle checks.

RQ2: Is the four-class partition a safety requirement or a naming choice? The partition is load-bearing: collapsing all four sources into one generic trusted context causes 3,593 false accepts out of 3,823 checker-denied events (94%). Even the weakest single-edge collapse (tool→agent) falsely accepts 1,928 events (50%). On 7 controlled workflow residuals, a post-hoc policy DSL that checks predicates without field ownership falsely accepts 7/7; splitting lifecycle state across separate guards falsely accepts 5/7.

RQ3: Does the same commit API work beyond tool calls? The `check_and_consume` transition generalizes across five boundary types: local env side effects, context placement, Skill instruction slots, delegation handoffs, and an OS-monitor-style replay backend. Across 38 checker-submitted boundary events, IntentCap allows 17 authorized effects and blocks 21 violations, with 0 unsafe executions or placements and 0 checker/monitor mismatches.

RQ4: Are leases auditable authority objects? All IntentCap policy families achieve oracle distance 0 across 24 adjudicated lease labels covering InjecAgent, MCPTox, and tau2 samples; every non-IntentCap baseline has positive distance over 144 scored policy rows.

4 Discussion

IntentCap asks a different question from adjacent work. EIM and bpftime ask how an extension can safely use host-provided resources [6]. ActPlane asks how policy can be enforced below the tool layer [5]. SkillGuard asks how a Skill package can be governed as a permission-bearing artifact [4]. IntentCap asks which future decisions a given context may influence, under a given user intent, and with what proof. That distinction keeps the LLM outside the trusted computing base, treats the current user goal as the root of authority, and makes context influence a least-privilege capability rather than an implicit side effect of putting text in the prompt.

Current evidence covers replay safety, four-class ablations, and local multi-boundary enforcement. End-to-end utility with a live user simulator, independent expert-oracle lease labels, and production OS-level enforcement remain open. Scaling the checker to real-time MCP broker integration and measuring approval burden are immediate next steps.

References

[1] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. 2024. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. In *Advances in Neural Information Processing Systems 37 (NeurIPS), Datasets and Benchmarks Track*. doi:10.48550/arXiv.2406.13352

[2] Model Context Protocol. 2025. Specification. https://modelcontextprotocol.io/specification/2025-06-18.

[3] OpenAI. 2026. Agent Skills. https://developers.openai.com/codex/skills. Accessed: 2026-07-02.

[4] Shidong Pan, Xiaoyu Sun, Tianyi Zhang, Dianshu Liao, Meixue Si, and Zhenchang Xing. 2026. SkillGuard: A Permission Framework for Agent Skills. arXiv preprint arXiv:2606.03024. doi:10.48550/arXiv.2606.03024

[5] Yusheng Zheng, Tianyuan Wu, Quanzhi Fu, Tong Yu, Wenan Mao, Wei Wang, Dan Williams, and Andi Quinn. 2026. ActPlane: Programmable OS-Level Policy Enforcement for Agent Harnesses. arXiv preprint arXiv:2606.25189. doi:10.48550/arXiv.2606.25189

[6] Yusheng Zheng, Tong Yu, Yiwei Yang, Yanpeng Hu, Xiaozheng Lai, Dan Williams, and Andi Quinn. 2025. Extending Applications Safely and Efficiently. In *Proceedings of the 19th USENIX Symposium on Operating Systems Design and Implementation (OSDI '25)*. USENIX Association. https://www.usenix.org/conference/osdi25/presentation/zheng-yusheng